Computer Architecture - 1DT093

ExamTag: 2024-Apr
Exam Date: 2024-04-05

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

1. How does the register file change after executing "addi R0, R1, 1"?
    1. R0 has R1+1
    2. R0 has R0+R1+1
    3. R1 has R0+1
    4. None of the above
2. What value should be put in X to load the word starting at address 26 into R8?
```
lw R2, 24(R0)
sll R2, R2, X
lw R3, 28(R0)
slr R3, R3, 16
or R8, R2, R3
```
    1. 8
    2. 16
    3. 24
    4. 32
3. Why do processors do all their computations on registers?
    1. Main memory is too slow
    2. Easier to address fewer locations
    3. A and B
    4. None of the above
4. How many fully usable general-purpose registers does MIPS have?
    1. 31
    2. 32
    3. 33
    4. More than 33
5. How many sequencing instructions (conditional or unconditional jumps) do you need to implement an if-than-else in MIPS assembly?
    1. 0
    2. 1
    3. 2
    4. 3
6. What are the source inputs for the bne instruction?
    1. 1 register, one immediate
    2. 2 registers, one immediate
    3. 3 registers
    4. 1 register, two immediates

2. ISA 2

1. A designer adds a "load big constant to register" instruction to MIPS. (lbc REG, immediate) This instruction uses a modified I-format encoding. How much does this speed up loading a single 32-bit immediate?
    1. No change
    2. 31% faster
    3. 65% faster
    4. 2x faster
2. What problem does the MIPS calling convention solve?
    1. Storing data before loading it into a register and after removing it from a register
    2. There are not enough registers for each procedure to have its own
    3. Procedures do not know which registers their caller and callees use
    4. How to access more than 32 words of data
3. Why is the instruction "bne R2, 5, loop" not possible in the MIPS ISA?
    1. Because I-type instructions don't have enough immediate field bits for two constants
    2. Because R-type instructions only have enough immediate field bits for one constant
    3. Because J-type instructions don't have any bits for registers
    4. Because I-type instructions need a constant offset, not a string like "loop"
4. This code will never finish. Why?

```
main:
        addi    R1, R0, 4
        addi    R2, R0, 0
        jal     sum_from_next_address
        j       finish
sum_from_next_address:
        addi    R1, R1, 4
        jal     load_data
        add     R2, R2, R3
        jr
load_data:
        lw      R3, 0(R1)
        jr
```

    1. It doesn't save temporary and saved registers
    2. It doesn't save the return address register
    3. It doesn't follow the register convention for argument and return registers
    4. All of the above
5. Why do we only need a few bits for relative branches?
    1. Branches are usually to nearby code
    2. Longer branches can be done with jump instructions
    3. The immediate (offset) is shifted and added to the PC+4 (base address) to generate the next PC. Thus even a few bits of immediate can generate the next PC
    4. All of the above

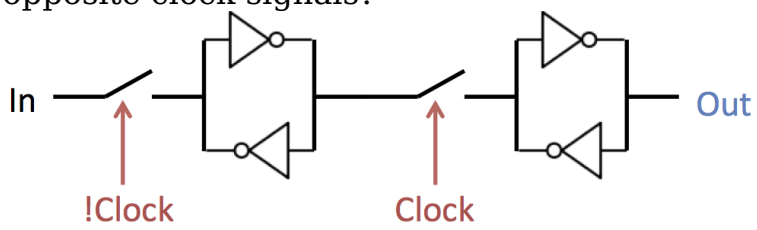
6. How does the processor know the instruction type?
    1. The binary code contains an instruction type bit for each instruction
    2. If there is an immediate in the instruction
    3. Looks at the opcode field
    4. If the instruction has 0, 2, or 3 register fields

3. Computer Arithmetic

1. What is the range of an 8-bit signed magnitude binary number?
    1. -64 to 64
    2. -64 to 63
    3. -128 to 127
    4. -127 to 127
2. Base-2 arithmetic is what is used in our MIPS processor: numbers are stored in binary (1s and 0s) and each binary digit represents twice the value before it. Another approach is to use Base-10 number representations, where we use 4 binary digits together to represent the numbers 0-9. (Representing 0-99 would require two sets of 4 bits, one for the one's digit and one for the ten's digit, and representing 0.0 to 9.9 would require the same number of bits.) We also need to build Base-10 adders that can do the right thing with this representation. Why would one want to use base-10 arithmetic in computers instead of base-2 arithmetic? (Hint: IBM actually makes processors that have hardware for Base-10. Why would they do that?)
    1. It is easier for humans to reason about because we think in Base-10
    2. It is easier for computers to work with because carries are simpler in adders
    3. It is easier to avoid rounding errors with values like money that are already in Base-10
    4. None of the above
3. Without normalization, how many ways are there to represent the number 3 with an seeeffff floating point format? (-1^Sx0.ffffx2^(eee-4))
    1. 1
    2. 2
    3. 3
    4. 4
4. How many bits are required to compute the following in two's compliment without an overflow? -8+8=0
    1. 4
    2. 5
    3. 6
    4. 7
5. What is the largest magnitude number you can represent with a two's complement fixed-point ddd.dd format?
    1. -4
    2. -3.75
    3. 3.75
    4. 4
6. Compute 1010 - 1101 for 4-bit two's complement numbers.
    1. Overflow
    2. 3
    3. -3
    4. -4

4. Logic

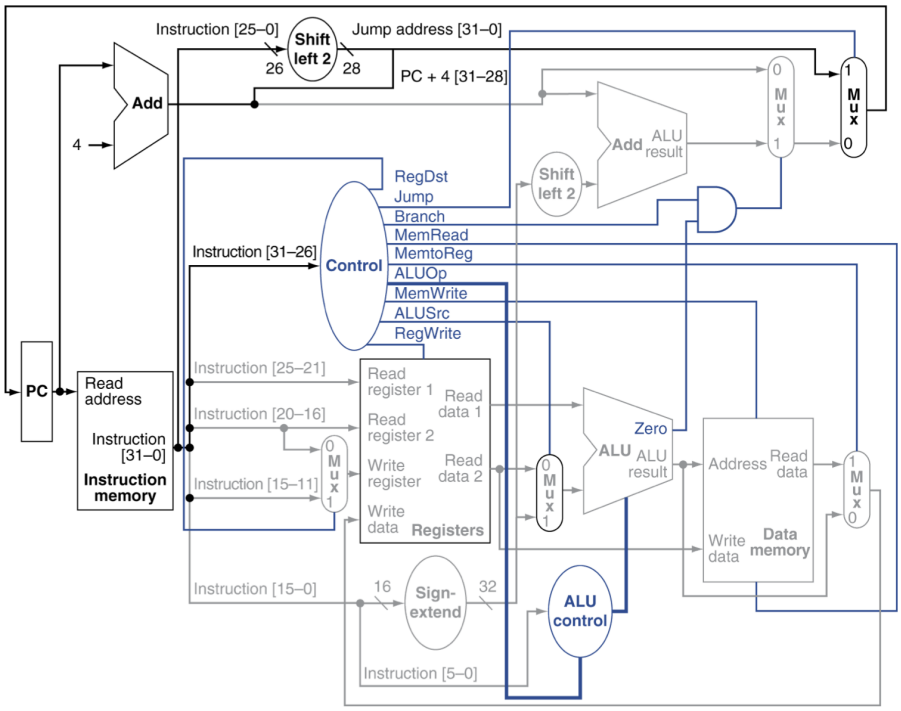
1. Why are DRAMs so much cheaper than SRAMs?
    1. They do not need decoders to select the data because they do not share output wires since the charge on the capacitors is so small
    2. Each bit is slower because it only has a small amount of charge from a capacitor which has to drive the output
    3. Each bit is smaller because it is stored in a trench deep down into the silicon instead of several transistors spread out on top
    4. All of the above
2. How many output bits does an adder have if it is adding two 8-bit numbers?
    1. 5
    2. 8
    3. 9
    4. Depends on whether the numbers are two's compliment or fixed-point
3. Up to how many inputs would a MUX have that has 3 select bits?
    1. 3
    2. 6
    3. 8
    4. 9
4. (!A AND B) OR (!B AND A) is the equivalent of which of the following?
    1. A AND B?
    2. A OR B
    3. A XOR B?
    4. A XNOR B
5. What changes from a simple latch when you have two latches back to back with opposite clock signals?



    1. The latch behaves the same but it takes longer for the data to get through
    2. The data is only updated when the clock goes from low to high
    3. The data only goes through the latch when the clock is low
    4. The latch can store a value when the clock is low
6. What is the current state in a counter circuit?
    1. The adder
    2. The next count
    3. The current count
    4. The clock

5. Processor Control and Datapath

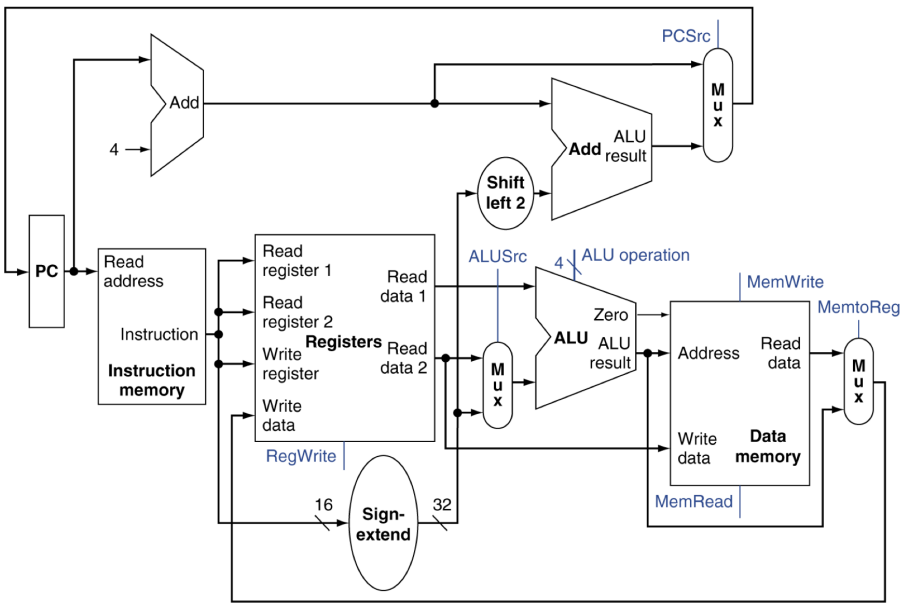
1. What do we need to add to the processor shown below to support the jr instruction?



   1. Another read and another write inputs to the Register File
   2. Path from the PC to the Register File and another ALU function to the ALU control
   3. Path from the Register File to the PC
   4. All of the above

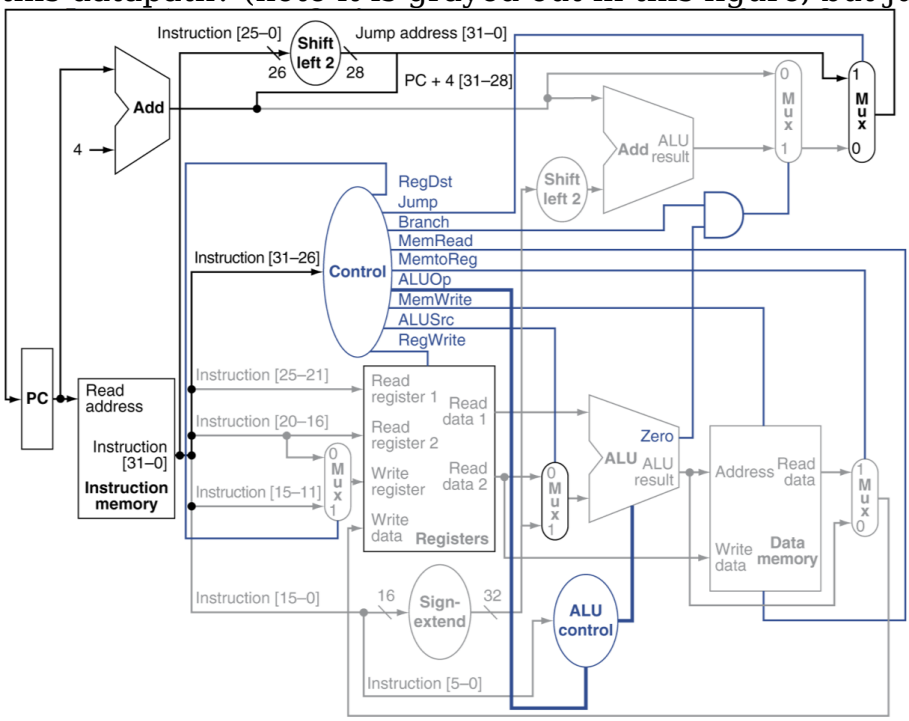
2. Why do we need two outputs from the control logic to properly handle bne and beq?
   1. We need one for branch taken and one for not-taken
   2. We need one for if Zero from the ALU is true and one for if it is false
   3. We need two to determine how to use the Zero output for the two branches
   4. We don't: we can do the logic with a single branch output from the control

3. What determines the value of the ALUSrc MUX?



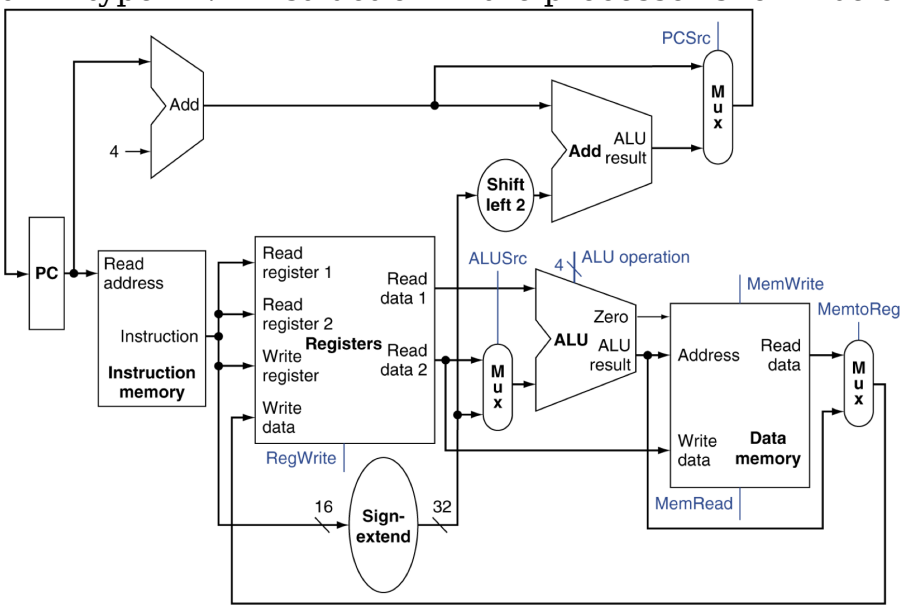
1. The instruction type: I=Read data 2, R=Sign-extend
2. The instruction type: I=Sign-extend, R=Read data 2
3. The ALU Zero output: True=Read data 2, False= Sign-extend
4. The value from Read data 2

4. Why do we have a MUX before the Write register going into the Register File in this datapath? (note it is grayed out in this figure, but just ignore that.)



1. To write PC+4 into the register file from JAL
2. To choose the write register address for I and R format
3. To choose the register to read out for JR
4. All of the above

5. What does the branch adder (the one with "shift left 2" going into it) compute on an R-type AND instruction in the processor shown below?



1. Nothing
2. The AND of two registers
3. The addition of a the next PC and an immediate
4. The addition of a register and an immediate

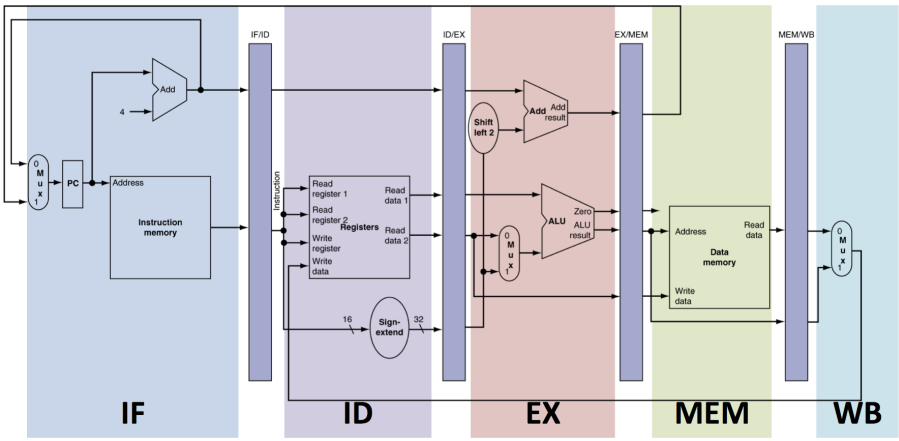
6. Why do we not have hazards in the single-cycle processor?
   1. Every instruction finishes before the next one starts so the register file always has the right data
   2. No instructions are executing in parallel so there are no conflicts between instructions
   3. Each instruction has access to the whole processor while it is executing so there is no fighting for resources
   4. All of the above

6. Pipelining

1. A processor takes 180ns for the longest instruction. The processor is pipelined with 6 (equal) stages using pipeline registers that take 10ns. How much more or less time does each instruction take compared to the un-pipelined processor? (You can assume there are N pipeline registers for N stages.)
    1. 140ns shorter
    2. same
    3. 10ns longer
    4. 60ns longer
2. What do you need to do to make sure a given pipeline can reach its full potential?
    1. Lots of stages
    2. Even stages
    3. High clock frequencies
    4. Keeping the pipeline full
3. What is the speedup you expect if you add 20 ideal pipeline stages to a processor, but you can only keep them full with instructions 25% of the time?
    1. 2.5x
    2. 5x
    3. 19.75x
    4. Need more information
4. What is the role of pipeline registers?
    1. They store the results of each stage so that the next stage can use them
    2. They allow the register file to store more values
    3. They reduce the number of NOPs we need to insert in the pipeline when executing a branch instruction
    4. All of the above
5. Does the pipelined processor execute instructions in an atomic and in-order manner?
    1. Atomic: yes, In-order: yes
    2. Atomic: yes, In-order: no
    3. Atomic: no, In-order: yes
    4. Atomic: no, In-order: no

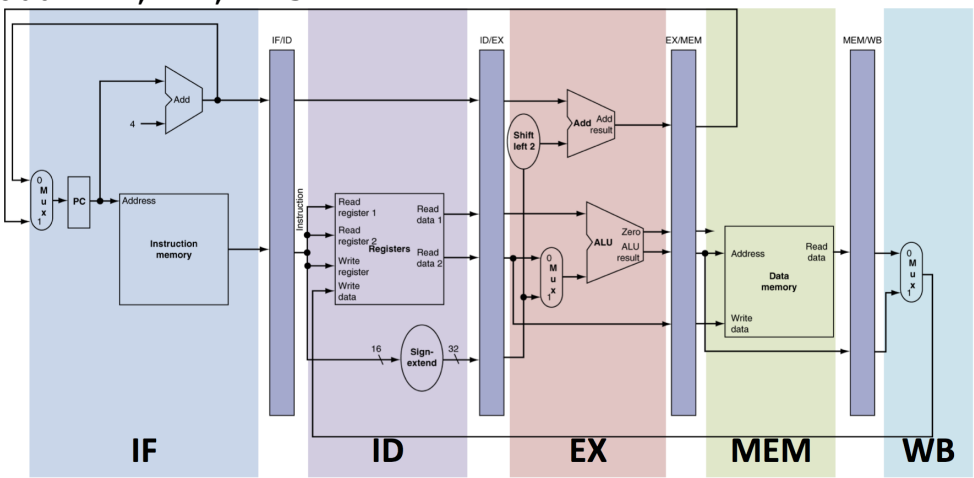
6. What value will be in R9 after this code finishes executing on the pipeline shown below?

```
R1=1, R4=1, R9=1
beq R1, R4, skip
addi R9, R0, 0
addi R9, R9, 2
addi R9, R9, 4
skip:
add R0, R0, 0
end:
```



1. 0
2. 1
3. 6
4. 12

7. Hazards

1. What is needed to resolve the following hazard?
   ```
   sub R4, R13, R14
   add R14, R4, R13
   ```
   

   1. Forwarding from WB to EX
   2. Forwarding from MEM to EX
   3. Double-pumping the RF
   4. Can't resolve: need to stall

2. What logic does the forwarding unit need to forward from MEM to EX? (Rs, Rt are the sources and Rd is the destination)
   1. (MEM.Rt == EX.Rd) OR (MEM.Rt == EX.Rt)
   2. (MEM.Rd == EX.Rs) OR (MEM.Rd == EX.Rt)
   3. (MEM.Rs == EX.Rt) AND (MEM.Rs == EX.Rd)
   4. (MEM.Rd == EX.Rd) AND (MEM.Rd != EX.Rt)

3. The instructions in the branch and load delay slots after the branch or load don't see the effect of the branch or load before they execute. Does this follow the ISA's promise of atomic and sequential execution?
   1. Yes
   2. Atomic yes, sequential no
   3. Atomic no, sequential yes
   4. Neither atomic nor sequential

4. What is the performance improvement (if any) of being clever (if you can) and filling the nop branch delay slot in this code?
   ```
   add R4, R5, R4
   add R3, R6, R5
   beq R3, R0, label
   nop
   sub R3, R3, R4
   ```
   1. No improvement
   2. Can't reorder
   3. 20% faster
   4. 40% faster

5. An 8-stage pipeline processor requires NOPs to avoid hazards. The compiler is aware of this and inserts appropriate NOP instructions into the code. What would happen if the processed code now is run on a single-cycle processor?
    1. The program would execute correctly
    2. Conditional branches would not work correctly, due to the zero output from the ALU
    3. The code would not work correctly because the NOPs are in the wrong place
    4. The code would not work correctly because there are too many NOPs

6. How many bubbles are needed in the pipeline to correctly execute the following code assuming no forwarding?
```
add r1, r2, r5
sub r2, r2, r3
add r3, r5, r5
ld r4, 0(r1)
beq r1, r2, done
add r1, r2, r3
sub r2, r2, r4
```
    1. 0
    2. 1
    3. 2
    4. 3

8. Branch Prediction and Exceptions and Interrupts

1. A 1-bit branch predictor has 4 entries and is direct-mapped based on the PC of the branch. What will happen with second branch in this loop?

   ```
   start:
     beq R1, R2, loop
       jal update
       j start
   loop:
     add R3, R1, R2
     beq R1, R2, start
   ```

   1. Predict fine
   2. Predict badly
   3. Depends on the initial values of R1 and R2
   4. Depends on how update changes R1 and R2

2. The following program has an interesting characteristic: branch A can be predicted nearly perfectly, and 90% of the time A is taken B is also taken. By itself B appears to be random. How could you build a branch predictor that would do a better job of predicting the B branch?

   ```
   if (A)
      do_something
   if (B)
      do_something
   ```

   1. Use more bits in the predictor
   2. Use the PCs of the past two branches to look up a prediction
   3. Use the outcomes of the past two branches to look up a prediction
   4. Use the PCs and outcomes of the past two branches to look up a prediction

3. What is the advantage of 2-bit branch predictors compared to 1-bit predictors?
   1. They change decision more slowly
   2. They learn faster
   3. They always make better predictions
   4. They use less logic

4. What can we do to avoid the penalty of the branch delay slot?
   1. Double-pump the register file
   2. Re-order instructions
   3. Put in a bubble
   4. Move the branch logic forward

5. What prediction strategy does our initial MIPS pipeline use?
   1. No strategy
   2. Predict taken
   3. Predict not taken
   4. Randomly choose taken/not taken

6. What is the slowdown for processors that does one instruction per cycle and has a branch predictor that is 85% accurate with a 75-cycle mis-prediction cost? (Assume 35% of instructions are branches).
    1. ~4x faster
    2. ~2x slower
    3. ~5x slower
    4. ~7.5x slower

9. Input/Output

1. What are the tradeoffs between polling and interrupts?
    1. Polling uses up little processor time but has a high latency
    2. Interrupts use up a lot of processor time but have a low latency
    3. Interrupts are good for low-latency I/O and polling for high-throughput I/O
    4. Interrupts use up little processor time but polling gives low latency
2. Why are solid-state (NVRAM/FLASH) drives so much faster than regular hard drives?
    1. Because they use no electronic components, only mechanical parts
    2. Because they use no mechanical parts, only electronic components
    3. Because they need to be constantly powered in order to retain data
    4. Because they use SRAM cells, similar to CPU caches
3. Why is it important that wires on a circuit board are the same effective length?
    1. So the signals arrive at the same time
    2. So the signals don't affect other wires
    3. So they pack densely on the board
    4. To reduce their capacitance
4. Why have busses become a problem in modern systems?
    1. Hard to make small wires
    2. Hard to connect many wires
    3. Hard to find space for long wires
    4. Hard to keep the data on the wires synchronized
5. How does differential signaling help in communications?
    1. Increases speed by sending two bits at the same time
    2. Puts the clock into the signal by using two opposite wires
    3. Improves synchronization by requiring only two wires
    4. Reduces the effect of noise on the wires by subtracting it out in the end
6. Which of the following is NOT a benefit of memory-mapped IO?
    1. Faster
    2. Uses regular instructions to access IO
    3. Unifies all the device's memories
    4. Enables the use of virtual memory to protect IO devices

10. Caches

1. What data will be in a 4-entry, direct-mapped, LRU cache with one word per line after the following words are accessed? 0, 1, 2, 7, 8, 4, 3, 1, 5, 6
    1. 1 3 5 6
    2. 3 4 5 6
    3. 0 1 3 6
    4. 1 4 5 6
2. An N-line, direct-mapped, write-through cache needs what in addition to the data?
    1. N valid bits, N dirty bits, N tags, N comparators
    2. N valid bits, N tags, N comparators
    3. N valid bits, N dirty bits, N tags, 1 comparator
    4. N valid bits, N tags, 1 comparator
3. What data will be in a 4-entry, fully-associative, LRU cache with one word per line after the following memory accesses? 0, 1, 2, 3, 4, 5, 3, 2, 1?
    1. 1 3 4 5
    2. 1 2 3 5
    3. 1 2 4 5
    4. 0 3 4 5
4. What data is evicted on a miss in a LRU cache?
    1. The data that has been in the cache for the longest time
    2. The data that has most recently been installed in the cache
    3. The data that has not been used for the longest time
    4. Misses do not evict data
5. A set-associative, write-back cache with a LRU replacement policy, sees the following accesses from the processor: `lw A` (miss), `lw B` (miss), `lw A` (hit), `sw A` (hit). What is changed in the cache on the last access?
    1. The LRU order of the sets, to update the most recently used one
    2. The dirty bit, to keep track of whether the data needs to be written back
    3. Which address bits are used to index the cache
    4. The valid bit, to indicate that the cache entry has data
6. Intel's Ivy bridge processors have a 256kB 16-way set associative L2 cache with a 64-byte cache line size. How many cache lines will each way have?
    1. 2048
    2. 512
    3. 256
    4. 64

11. Virtual Memory

1. What is not a benefit of having a virtual address to physical address translation?
    1. Memory protection
    2. Using disk as memory
    3. Faster cache accesses
    4. Using fragmented memory
2. What happens when process 1 loads address 10 given the page tables below?

process	VA	PA	valid	dirty
1	3	4	1	1
1	6	5	1	0
0	3	6	1	0
0	10	7	1	1

    1. It loads physical address 7 from DRAM
    2. It loads physical address 10 from DRAM
    3. It causes a page fault and the OS loads the data for physical address 7 from disk before returning it to the program
    4. It causes an protection violation and the OS kills it
3. Which of the following statements is true about virtual memory?
    1. VM provides security by preventing programs from changing each other's data, flexibility by letting programs use as much memory as is physically available, and performance by making DRAM accesses faster
    2. VM provides security by preventing programs from changing each other's data, flexibility by letting programs use as much memory as is physically available, and performance by allowing programs to share data without copying it.
    3. VM provides security by preventing programs from changing each other's data, flexibility by letting programs use more memory than is physically available, and performance by allowing programs to share data without copying it.
    4. All statements above are true.
4. What is the TLB?
    1. A cache
    2. A predictor
    3. A tag
    4. A page table
5. What is generally slower than a page fault to a hard disk?
    1. Accessing data over the network
    2. Switching to another program
    3. The human using the computer
    4. All of the above
6. Why would we prefer a single-level page table rather than a multi-level one?
    1. To reduce the total size of the page table
    2. To make translations faster
    3. To reduce the in-memory size of the page table
    4. None of the above

12. Parallelism

1. The cache for processor 2 wants to load an address. What does the cache for processor 1 need to check to see if it needs to write back data in its own cache and tell processor 2's cache to wait for it?
    1. If it has the dirty line (tag and dirty bit)
    2. If it has the line (tag and valid bit)
    3. If it has the line (tag and valid bit) and it is not dirty
    4. If it has the line (tag and valid bit) and it is dirty
2. Why did manufacturers start making multicore processors?
    1. They can't increase the clock speed due to power and heat
    2. They can't build circuits that run faster due to having too small transistors
    3. It is easier to write software for parallel processors than for a single fast processor
    4. All of the above
3. What speedup would we expect from a triple-issue pipeline that can do 1 load/store instruction and 2 other instructions at the same time for a program that is 1/3 load/store instructions?
    1. Less than 3x
    2. 3x
    3. More than 3x, but less than 6x
    4. More than 6x
4. What could happen if we do *not* use atomic instructions while implementing locks?
    1. Hazards in the pipeline.
    2. The appearance of shared data in the cache that shouldn't be there.
    3. The data could be changed during the process of acquiring the lock.
    4. A segmentation fault.
5. How much faster can my program run if I have 9000 cores and 20% of the program cannot be parallelized?
    1. 5x
    2. 1800x
    3. 7200x
    4. 9000x
6. My laptop has 2 CPU processor cores that run at 2GHz (billion cycles per second) and can each do 4 instructions per cycle and a GPU (graphics processor) with 16 processor cores that run at 1GHz and each do 2 instructions per cycle. You can use either the CPU or the GPU at any given time. What is the fastest rate I can run a program that is 50% parallelizable in GIPS (giga (billion) instructions per second)?
    1. 12 GIPS
    2. 17 GIPS
    3. 20 GIPS
    4. 32 GIPS